PROGRAM SUMMARY

Program title: readcon-core
CPC Library link to program files: (to be added by Technical Editor)
Developer's repository link: https://github.com/lode-org/readcon-core
Licensing provisions: MIT
Programming language: Rust, C, C++, Python, Julia, Fortran 2008
Supplementary material: specification, grammar, conformance corpus, and the tables behind the measurements

Nature of problem: Rare-event campaigns stop and resume from a stored geometry. The formats in common use each drop part of the resume state: a per-axis mask, a persistent atom identifier, the velocity time unit, binary64 coordinates, or the periodic flags. Readers of one format also disagree with each other, so the resumed search runs on a different surface.

Solution method: CON 3.1 is a versioned grammar. A reader of a known major accepts a later minor, keeps unknown keys, and rejects an unknown major. Every velocity block declares its time unit. The default writer emits the shortest decimal that parses back to the same binary64 value. A corpus of legal and illegal frames decides what a conforming reader accepts. One Rust core sits behind one C interface, and eOn 3.6.0 reads and writes CON through it. The companion store keeps that text, indexes composition, energy and forces, recognises gzip and zstd wrappers of the same text, and ingests frames from several MPI ranks. Its map starts at 2 GiB, and the caller can raise that limit.

Additional comments including restrictions and unusual features: A CON 3.1 frame stores Cartesian coordinates, the cell lengths and angles, the periodic flags, a per-axis mobility mask and a persistent atom identifier. From minor 1, a velocity block names its time unit. The same row layout carries forces, per-atom energies, charges, spins, magnetic moments, displacements and spreads. The JSON header carries the grammar version, and it carries an energy, a potential name and band indices when the writer recorded them. gzip and zstd wrappers of that text are part of the grammar. The frame does not store the displacement history of a quasi-Newton update or of a FIRE step, so an optimiser that needs those vectors takes a different step after a resume. A band is a sequence of frames. It is not one frame that holds the optimiser state. Files written by earlier eOn releases are grammar version 1: column 4 is 0 or 1 and the JSON line is absent. A 3.1 reader accepts them under that column rule, and a 3.1 writer emits the versioned header. The memory-mapped store keeps each CON text and indexes on composition, energy and maximum force. The map starts at 2 GiB and the caller can raise it. Thread, peer and MPI timings used readcon-core 0.17.0. The store timings used

readcon-core 0.16.2 and readcon-db 0.1.8. The release version and its DOI are the data-availability statement.

CON 3.1: a versioned restart state for saddle searches

Rohit Goswami[a,1]

[a] *TurtleTech ehf., Reykjavík, Iceland*

Abstract

A saddle search resumes from a geometry file, so that file is the restart state of the algorithm. The resume needs a per-axis mask, a persistent atom identifier, velocities in a declared time unit, coordinates that round-trip in binary64, and a versioned grammar. On one masked Pt(100) frame, every common format tested here drops at least one of those fields. An ASE POSCAR round trip drops the adatom's height constraint, and the resumed climbing-image band finishes 70.2 meV below the uninterrupted barrier of 0.644561366 eV. Five extended XYZ readers return four answers for one file. CON 3.1 is the versioned grammar of the text eOn already writes, checked by a conformance corpus and implemented once: a Rust core behind a C interface for C, C++, Python, Julia and Fortran. A harness of twenty-five writer-reader pairs among those languages resumes that band at the uninterrupted barrier. A warm read of 10,000 frames of 218 atoms, recorded with readcon-core 0.17.0, takes 12.6 μs per frame. On that file a chemfiles read of the XYZ coordinates is 2.15 times as fast. h5py is 67.1 times as fast when the timed object is one numeric array, not the mask, the identifier, the cell and the periodic flags.

Keywords: restart, specification, saddle search, conformance, nudged elastic band

1. Introduction

Rare-event methods turn a potential energy surface into rates. Transition state theory [1, 2, 3] reduces kinetics to saddle points. These are located by the nudged elastic band [4, 5, 6, 7], by minimum-mode followers [8, 9, 10], and by string methods [11, 12]. The results are chained into adaptive kinetic

Email address: rgoswami@ieee.org (Rohit Goswami)
[1]ORCID: 0000-0002-2393-8056.

Monte Carlo and accelerated dynamics [13, 14, 15, 16]. None of these is a single calculation. A long-timescale run in eOn [17, 18] spawns thousands of saddle searches, often on volunteer or distributed resources [19, 20]. It restarts them, and it passes geometries between Python drivers [21, 22], C++ engines and Fortran electronic structure codes [23]. The geometry file is therefore not an export format. It is the *restart state* of the algorithm.

That restart state has requirements that general-purpose formats were not designed for. A surface saddle search fixes the slab and lets an adatom move only along the surface normal. It therefore needs mobility per atom *and per axis*. Mode following and AKMC bookkeeping need atom identity that survives reordering. Dynamics-based steps need velocities whose unit is declared. eOn's natural time unit, $(\text{amu}\,\text{Å}^2/\text{eV})^{1/2}$, is 10.180505710759414 fs for the CODATA 2018 constants [24]. The square of that value, 103.64269652680504, is the factor in the kinetic energy when the same numbers are read as Å/fs. A converged band restarted from rounded coordinates is no longer converged, so coordinates must round-trip in binary64 [25, 26, 27, 28]. Finally, two readers given the same bytes must agree, which needs a versioned grammar rather than a convention.

Existing formats meet some of these requirements, but none meets all of them. Extended XYZ carries per-axis masks only through an ASE-specific column. It writes coordinates at eight decimals by default, which moves positions by up to 10^{-9} Å on a round trip, and it has no version or velocity unit. H5MD [29, 30] records identity, velocities and box, but it has no per-axis constraint field and treats units as optional attributes. TNG [31, 32] and NetCDF trajectories [33] target compressed MD output, not optimiser restarts. ASE's own trajectory file is lossless within ASE, but it is defined by its Python implementation and cannot be read from the C++ and Fortran codes that do the work. Readers of the original CON format drift too. We found that the ASE copy of the eOn writer currently fails outright on per-axis constraints, and that a POSCAR round trip silently drops them. Toolkits such as chemfiles, MDAnalysis, Open Babel and IOData [34, 35, 36, 37] widen *access*, but they inherit whatever each format leaves undefined.

1.1. Contribution

We do not propose another container. We make the format eOn already uses into a *specification with a test suite* and give it a single implementation:

1. **CON 3.1**, a versioned grammar that specifies per-axis mask bits, a

persistent atom identifier, velocities in a declared unit, and shortest round-trip binary64 output [38, 39].

2. **An executable conformance corpus**. Any reader, ours or not, can be checked against it. Agreement between readers is then something we test, not something we assume.
3. **One core, many bindings**. A Rust library behind a C ABI serves C++, Python, Julia [40] and Fortran. Per-language behaviour cannot drift, because there is only one parser. This follows the precedent set by chemfiles [34]. eOn 3.6.0 reads and writes through it.

We show each failure above as a reproducible case and show that CON 3.1 removes it. We then demonstrate bit-exact restart of a masked, asymmetric NEB across languages. Throughput comes last. We report it against the peer readers, with the cases where readcon is slower stated plainly [41, 42]. Section 2 measures the five requirements on one frame, Section 3 states the grammar, and Section 4 is the corpus and the extended XYZ disagreement. Section 5 is the reader and the store. Section 6 resumes one band through every route we completed, and Section 7 reports threads, peers, MPI and the store. Those timings name the build that was clocked.

2. Requirements and the failure each one closes

Table 1 states five requirements. Each is a field a later step reads. The frame stores the geometry and its labels. It does not store the short history of a quasi-Newton update [43, 44] or of a FIRE step [45]. Section 6 measures that difference on one band.

R1 is a mask for each Cartesian axis of each atom. R2 is an integer stored on the row, so a later frame finds the same atom after the file groups rows by element. R3 is a time unit written in the file. R4 is coordinates, cell and periodic flags that read back as the values written. R5 is a grammar with a major and a minor, a rule for what a later minor adds, and a corpus that decides disputes.

2.1. One frame through each format

The frame is a Pt(100) slab with one adatom: the bottom atoms held on every axis, the adatom held in z, identifiers that are not the row order, and velocities in the natural unit. Section S7 gives the coordinates, the identifiers and the grading rules. Each row of Table 2 writes that frame with one library

Table 1: Requirements on a restart state, the step that reads each field, and the failure measured when the field is missing.

Requirement	Step that reads it	Failure measured here
R1, per-axis mask	Surface search, prefactor	Height constraint dropped; barrier $-70.2\,\mathrm{meV}$
R2, persistent id	Band and event table	Id lost on completed foreign round trips
R3, declared time unit	Dynamics	Kinetic energy scaled by 103.642696
R4, binary64 geometry, cell, periodicity	Resume of a search	10^{-9} to 10^{-6} Å, or periodicity rewritten
R5, versioned grammar	Any second reader	One file, four answers from five readers

and reads it with another. The libraries are ASE 3.29.0 [21], chemfiles 0.10.4 [34], MDAnalysis 2.10.0 [35, 46], and h5py 3.16.0 where the route needs it. POSCAR selective dynamics is the per-axis mask in the VASP ionic restart [47, 23].

The CON 3.1 row keeps every field. Extended XYZ written and read by ASE returns the mask, the identifiers and the time unit because both sides share ASE's column names and ASE's internal unit. The extended XYZ grammar does not define those names, so chemfiles drops the mask and MDAnalysis drops the mask and the identifiers. The ASE writer prints a fixed number of decimals and the coordinates move by up to 4.7×10^{-9} Å. chemfiles writing the same frame moves them by 4.9×10^{-6} Å and returns every axis periodic. MDAnalysis returns single-precision coordinates, a shift of 2.2×10^{-7} Å. The ASE trajectory keeps the mask and the coordinates exactly and drops the identifier. Its time unit is the library's unit, which the file does not name. The ASE POSCAR writer turns the held slab into a different mask and writes the adatom free. POSCAR has no atom identifier and no open axis. H5MD written by MDAnalysis declares the velocity unit, writes no mask and no identifier, stores single-precision coordinates, and marks the open z axis periodic [29]. An extra array carries a mask only when every reader agrees on its name and type. Section 6 attaches that array and shows which readers return it. The ASE eon reader rejects a CON frame that contains a velocity block. Given coordinates alone it freezes every masked

Table 2: One restart frame through each writer and reader. The frame is the Pt(100) slab with one adatom: the bottom layer is held on every axis, the adatom is held in z, ids are not the row order, and velocities are in the natural unit. R1 to R4 are measured on the round trip. "Kept" means the field came back. "Convention" means it came back only because writer and reader share one library's column name or internal unit, which the format does not define. "Fixed by format" means the format's own definition sets the unit without writing it in the file. R4 gives the largest coordinate change in Å when the coordinates are not bit-identical. Periodicity is the slab's xy-periodic, z-open flags after the read. R5 is the version rule in the format's published definition. "Error" rows failed before any field could be compared; the text gives the failure.

Format	Writer	Reader	R1 mask	R2 id	R3 unit	R4 coordinates	Periodicity	R5 version
CON 3.1	readcon-core	readcon-core	kept	kept	declared	exact	kept	major.minor
CON 3.1	readcon-core	ASE eon reader	error	error	error	error	error	major.minor
CON 3.1, coordinates only	readcon-core	ASE eon reader	full freeze	lost	lost	exact	none	major.minor
legacy CON	ASE eon writer	readcon-core	error	error	error	error	error	none
extended XYZ	ASE	ASE	convention	convention	convention	4.7×10^{-9}	kept	none
extended XYZ	ASE	chemfiles	lost	convention	lost	4.7×10^{-9}	not returned	none
extended XYZ	ASE	MDAnalysis	lost	lost	lost	2.2×10^{-7}	not returned	none
extended XYZ	chemfiles	ASE	lost	lost	lost	4.9×10^{-6}	all periodic	none
ASE trajectory	ASE	ASE	kept	lost	convention	exact	kept	code only
POSCAR	ASE	ASE	changed	lost	fixed by format	4.5×10^{-17}	all periodic	none
H5MD	MDAnalysis	MDAnalysis	lost	lost	declared	2.2×10^{-7}	all periodic	major.minor

atom on every axis, drops the identifiers, and returns no periodic axis. The ASE eon writer cannot express a per-axis constraint, so that route produces no file. Table S9 places the same checkpoint next to trajectory containers and structure archives.

3. The CON 3.1 specification

A CON file is a sequence of text frames. Listing 1 is one complete nine-atom frame. The survey frame of Table 2 uses the same columns and different identifiers, listed in Section S7.

Listing 1: A complete nine-atom Pt frame in CON 3.1. Line 2 declares the version, the units and the periodicity. Every atom has a coordinate row and a velocity row.

```
Pt(100) 2x2x2 with one Pt adatom
{"con_spec_minor":1,"con_spec_version":3,"pbc":[true,true,false],"sections
    ":["velocities"],"units":{"energy":"eV","length":"angstrom","mass":"
    amu","time":"(amu*angstrom^2/eV)^0.5"}}
5.5437171645025325e0 5.5437171645025325e0 1.396e1
9e1 9e1 9e1


1
9
1.95084e2
```

```
Pt
Coordinates of Component 1
1.3859292911256331e0 1.3859292911256331e0 6e0 7 0
4.1577878733769e0 1.3859292911256331e0 6e0 7 1
1.3859292911256331e0 4.1577878733769e0 6e0 7 2
4.1577878733769e0 4.1577878733769e0 6e0 7 3
0e0 0e0 7.96e0 0 4
2.7718585822512662e0 0e0 7.96e0 0 5
0e0 2.7718585822512662e0 7.96e0 0 6
2.7718585822512662e0 2.7718585822512662e0 7.96e0 0 7
1.3859292911256331e0 1.3859292911256331e0 9.76e0 4 8

Pt
Velocities of Component 1
1.4685859972256804e-3 6.275175116609037e-4 -1.8115929554652053e-3 7 0
2.1e-3 1.2e-3 -0.4e-3 7 1
0.8e-3 -1.5e-3 0.3e-3 7 2
-1.1e-3 0.6e-3 1.4e-3 7 3
1.0e-3 -0.7e-3 0.2e-3 0 4
-0.5e-3 1.3e-3 -0.9e-3 0 5
0.4e-3 0.9e-3 1.1e-3 0 6
-1.6e-3 -0.2e-3 0.5e-3 0 7
3.748769356553213e-3 2.43327727576049e-3 -1.3238979663774354e-3 4 8
```

Line 1 is free text. Line 2 is one JSON object [48]. Lines 3 and 4 are the cell lengths and the angles in degrees. The type count, the per-type counts and the masses follow. Each coordinate row is x, y, z, the mask and the identifier. Lengths and angles rebuild the box with a along x and b in the xy plane,

$$\mathbf{a} = a\,\hat{\mathbf{x}}, \qquad \mathbf{b} = b\cos\gamma\,\hat{\mathbf{x}} + b\sin\gamma\,\hat{\mathbf{y}}, \tag{1}$$

after which $\mathbf{c}$ is fixed by α and β so the cell is right-handed [49]. When line 2 carries lattice vectors, those rows are the cell. A missing periodicity flag means periodic on every axis.

R1, the mask.. Column 4 is the integer of Table 3. Mask 0 is free, mask 4 fixes z only, and mask 7 fixes every axis. A float spelling, a sign, or a value outside 0 to 7 is an illegal flag. Under version 1 the column is 0 or 1, so masks 4 and 7 cannot be stored as written.

R2, the identity.. Column 5 is an integer from 0 through $2^{64} - 1$. Writers emit identifiers unique within a frame. A missing identifier column takes the zero-based row index.

Table 3: Mask bits from version 2. Version 1 stores 0 (free) or 1 (fixed on x, y and z) in the same column.

Bit	Axis	Value
0	x	1
1	y	2
2	z	4

R3, the time unit.. From minor 1, a velocity block declares `units.time`. The natural unit is spelled `(amu*angstrom^2/eV)^0.5`. The reference reader converts that spelling to 10.180505710759414 fs for the CODATA 2018 constants [24]. The square of the printed value is 103.64269652680504. Versions 1, 2 and 3.0 are read in the natural unit. An fs label on those older files is not trusted: producers of that era wrote natural-unit numbers under an fs label [18].

R4, the numbers.. Readers convert each finite decimal to binary64 with correct rounding [25, 26]. The default writer prints the shortest decimal that parses back to the same value [28, 39]. Six fixed decimals are an optional writer mode. gzip and zstd wrap the same bytes, and a reader recognises the wrapper by content [50, 51]. A conforming reader accepts the compressed file and returns the same frames.

R5, the version.. Table 4 lists the reserved keys of the JSON line. A reader accepts a later minor of a known major, keeps an unknown key on rewrite, skips an unknown section whose width is declared, and rejects an unknown major. H5MD uses the same split for its version pair [29]. Table 5 is the history of the grammar. Table 6 is the sections reserved at minor 1. The reader rules are Table S1. The grammar of one frame is the listing in Section S15. The prose specification wins where the listing and the prose disagree.

3.1. What the text costs

eOn has written this text since its first releases [17]. Those campaigns store grammar version 1: column 4 is 0 or 1 and there is no JSON line (Table 5). A 3.1 reader accepts that file under the version-1 column rule. A 3.1 writer emits the versioned header, which is a different file. The bytes do not depend on endianness [20]. A diff of a text checkpoint shows a changed mask or identifier. The JSON line takes a new key without a new grammar, and a reader keeps a key it does not know [48].

Table 4: Reserved metadata keys of the JSON header line.

Key	Type	Key	Type
`con_spec_version`	int	`neb_bead`	int
`con_spec_minor`	int	`neb_band`	int
`section_arity`	object	`convergence_fmax`	number
`sections`	array	`convergence_energy`	number
`validate`	bool	`converged`	bool
`generator`	string	`fmax`	number
`units`	object	`bonds`	array
`pbc`	array	`storage_dtypes`	object
`lattice_vectors`	array	`chemfiles_atom_names`	array
`energy`	number	`chemfiles_atom_types`	array
`potential`	object	`chemfiles_atom_properties`	array or object
`frame_index`	int	`time`	number
`timestep`	number		

Table 5: CON versions. The date is the day the rule was fixed.

Version	Date	Change
1	eOn text	Column 5 present. Column 4 is 0 or 1. No JSON line.
2	2026-03-25	A JSON header line, identifiers, bit masks, declared sections.
3.0	2026-06-27	Length and energy units required. No minor.
3.1	2026-10-11	Minor 1. A velocity block declares the time unit.

Table 7 is the byte cost of one 218-atom frame and of that frame repeated 10,000 times, raw and after gzip at level 6. The repeated file is the same text, so its gzip column is smaller than the gzip column of a trajectory whose frames differ. One CON frame is 41.99 bytes per atom, against 83.64 for extended XYZ written by ASE with the mask column, 314.39 for one H5MD frame written by MDAnalysis, and 95.40 for one H5MD frame written by h5py with binary64 positions, a mask dataset and an identifier dataset. At 10,000 copies the raw CON cost stays 41.99 bytes per atom. H5MD amortises its container and falls to 12.61 and 24.04 bytes per atom. gzip of the repeated CON file reaches 0.39 bytes per atom. H5MD is the record for a long trajectory [29]. CON is the record for the checkpoint, where one frame has to keep the mask, the identifier and the unit.

Table 6: Reserved sections at minor 1. Each data row carries its values, then the mask and the identifier.

Section	Values	Rule
Velocities	$v_x\, v_y\, v_z$	Time unit from minor 1
Forces	$f_x\, f_y\, f_z$	
Energies	one value	Name `energies`
Charges	one value	
Spins	one value	
Magmoms	$m_x\, m_y\, m_z$	
Displacements	$d_x\, d_y\, d_z$	`units.length`
Spreads	$s_x\, s_y\, s_z$	`units.length`, each at least 0

Table 7: Bytes per atom of the 218-atom frame, alone and repeated 10,000 times, raw and after gzip at level 6. CON 3.1 is the default readcon-core writer, extended XYZ the ASE writer with the mask column, H5MD (MDAnalysis) its writer in single precision, and H5MD (h5py) binary64 positions with mask and id datasets. The repeated file compresses better than any real trajectory would.

	One frame		10,000 frames	
Format	raw	gzip	raw	gzip
CON 3.1	41.99	6.07	41.99	0.39
extended XYZ (ASE)	83.64	6.65	83.64	1.13
H5MD (MDAnalysis)	314.39	9.04	12.61	0.12
H5MD (h5py)	95.40	9.66	24.04	0.16

4. Conformance and drift

The corpus is the executable form of the grammar. It holds 10 legal files and 8 illegal files, split as in Table 8. A conforming reader accepts the first group and rejects the second. The conformance goldens are Python, C, Fortran and Julia. There is no C++ golden. C++ is in the resume harness of Section 6, which is a separate measurement from the corpus.

Table 9 applies R3 and R5 to seven headers of one velocity frame. Python and C follow the rule on each row. They reject a minor-1 velocity block with no time unit, an unknown major, a missing version and a mask of 9, and they keep an unknown key from a later minor. The ASE eon reader has no version rule. Given the coordinate block, it accepts all seven, including the illegal mask and the unknown major.

Table 8: Conformance corpus. Counts are the committed manifest summary.

Category	Files	What a reader must do
Header and units	2	Accept a version-2 header and a version-3 units line.
Column-4 masks	5	Accept version-1 full freeze and version-2 per-axis flags.
Empty frame	3	Accept a type count of zero.
Illegal	8	Reject the file.

Table 9: Header variants of one velocity frame. Rule is what the specification requires. The ASE eon reader is given the coordinate block only, because it stops at a velocity section.

Variant	Rule	Python	C	ASE eon
3.1, natural unit declared	accept	accept	accept	accept
3.1, no time unit	reject	reject	reject	accept
3.0, labelled fs	accept (natural)	accept	accept	accept
3.2, unknown key	accept, keep	accept, kept	accept	accept
4.0	reject	reject	reject	accept
missing version	reject	reject	reject	accept
3.1, mask 9	reject	reject	reject	accept

Extended XYZ has a published layout and parsers in several languages, so it is the text format to compare on R5. Table 10 gives five readers seven three-atom files. The readers are ASE 3.29.0, libAtoms 0.4.6 through its C path and its Python path, chemfiles 0.10.4, and MDAnalysis 2.10.0. On the reference frame the five readers return four outcomes: ASE and the libAtoms C path accept it, the libAtoms Python path changes the mask booleans, chemfiles moves one coordinate by one unit in the last place and returns no mask, and MDAnalysis returns single precision with no mask and no identifiers. The correctly rounded binary64 value of the coordinate 2.7718585822512662 has bit pattern `40062cc4313f3fb9`; chemfiles returns `40062cc4313f3fb8` (Section S7). A Fortran exponent `6.0d0` is read by the libAtoms C path, returned as NaN by the libAtoms Python path, and rejected by the other three. A plain comment line is non-periodic to ASE, periodic to the libAtoms C path, and rejected by the libAtoms Python path. The readers follow their own documentation. The grammar does not decide these cases, and the formats share no corpus that would.

Table 10: Five extended XYZ readers on seven three-atom files. Each file departs from the reference frame in one respect. ✓: positions correctly rounded, periodicity as written, mask and ids returned as written. E: the read failed. u: a coordinate one unit in the last place from the correctly rounded value. f: single-precision coordinates. n: a coordinate returned as NaN. p: periodicity differs from the file, or from the grammar's default when the key is absent. b: the mask came back with different booleans. m, i: the reader returns no mask, or no ids.

File	ASE	libAtoms C	libAtoms Py	chemfiles	MDAnalysis
reference frame	✓	✓	b	um	fmi
no pbc key	✓	✓	b	um	fmi
plain comment line	✓	p	E	u	f
Fortran exponent `6.0d0`	E	✓	nb	E	E
`True`/`False`	✓	✓	b	um	fmi
quoted value with `=`	✓	✓	b	um	fmi
`units` key	✓	✓	b	um	fmi

5. Reference implementation

The reference reader and writer is the Rust core behind the C interface. The concept record covers the line of releases [52]. Figure 1 is the layering. One Rust core parses and writes. One C interface is the waist every other caller uses. C++ wraps it for eOn 3.6.0. Python exposes array views [53]. Julia calls the C interface [40]. The resume harness calls that interface from Fortran 2008. chemfiles took the same route for trajectories: one core and several language bindings [34]. Decimal fields go through one correctly rounded conversion on that shared path [28].

The caller owns a frame handle and frees it. Structure-of-arrays accessors return a pointer into that frame. Absent numeric metadata is a NaN sentinel and is not written back. The reader records the byte offset of each frame. The scan that finds those offsets is serial, because each start depends on the counts before it. Each frame is then parsed on its own, and the frames return in file order.

Listing 2: A round trip in Python. The default printer preserves binary64. Six decimals are a separate argument.

```
frames = readcon.read_con("slab.con")
readcon.write_con("rt.con", frames)
```

readcon-db is the campaign store for that text [54]. It writes each CON frame into a memory-mapped store, posts an xxHash3-128 of the text, and

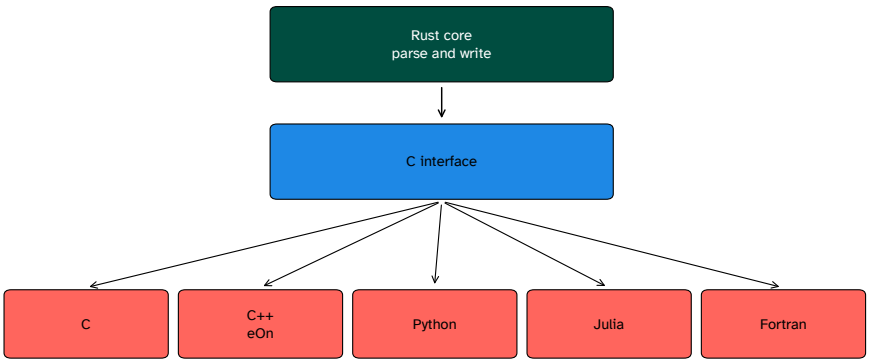


Figure 1: The Rust core parses and writes CON. The C interface is the waist. eOn 3.6.0 calls the C++ wrapper.

keeps secondary indexes on atom count, composition, energy, maximum force and section presence. A select touches the index and then the frames the index names. Identical texts share one hash entry, and each frame keeps its own stored text, so the stored blob bytes equal the raw file. gzip and zstd remain properties of the CON bytes, recognised by content before ingest. The MPI paths are a single writer, which inserts in global frame order, and a sharded ingest. Section 7.3 measures both. The map starts at 2 GiB. The caller can raise that limit, and a write that fills the map grows it and retries. Section 7 names the versions that were clocked.

6. Case studies

Every route below uses one protocol. An EMT Pt(100) slab, three layers of 3×3, has its bottom layer held on every axis and one adatom held in z. The adatom hops between neighbouring hollows. Endpoints were relaxed to $0.002\,\mathrm{eV/Å}$. A climbing-image band of five movable images uses the improved tangent [5, 6] and fixed-step steepest descent on the projected force. Steepest descent stores no step history, so a difference after the resume comes from the file. The band is interrupted after 25 steps, written, read back, and resumed to $0.01\,\mathrm{eV/Å}$. Uninterrupted, it converges in 60 steps with barrier $0.644561366\,\mathrm{eV}$ and 56 free modes on the middle image. Table 11 is every completed route. Figure 2 is three of those bands. Table S4 is the harness of 25 writer-reader pairs among Python, C, C++, Fortran and Julia.

Table 11: Climbing-image band of the EMT Pt(100) hop, stopped after 25 steepest-descent steps, written and read back, then resumed to 0.01 eV/Å. File is the largest coordinate change on that round trip. Modes are the free Cartesian degrees of freedom of the middle image (56 when the masks survive). $\Delta E^{\ddagger}$ and the displacement are against the uninterrupted band, whose barrier is 0.644561366 eV. [a]Not converged at 4000 steps. The 25 language pairs are Table S4.

Route	File (Å)	Modes	Steps	$\Delta E^{\ddagger}$ (eV)	Max displacement (Å)
uninterrupted	–	56	60	0	0
CON 3.1, shortest round trip	0	56	60	-5.68×10^{-14}	2.66×10^{-15}
CON 3.1, six decimals	4.90×10^{-7}	56	60	-4.14×10^{-7}	6.22×10^{-7}
CON 3.1 read by the ASE eon reader	0	54	4000[a]	-3.52×10^{-1}	5.43×10^{0}
same, periodicity restored by hand	0	54	127	-2.93×10^{-6}	4.21×10^{-2}
extended XYZ (ASE)	5.00×10^{-9}	56	60	1.09×10^{-9}	4.94×10^{-9}
ASE trajectory	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
POSCAR (ASE)	4.90×10^{-17}	57	98	-7.02×10^{-2}	1.60×10^{-1}
legacy CON (ASE eon writer)	no file: a per-axis constraint is rejected				
ASE trajectory, id array	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
H5MD MDAnalysis to MDAnalysis	4.60×10^{-7}	84	97	-7.07×10^{-2}	1.43×10^{-1}
H5MD MDAnalysis to h5py	4.60×10^{-7}	56	60	-1.98×10^{-7}	3.67×10^{-7}
H5MD h5py to h5py	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
H5MD h5py to MDAnalysis	4.60×10^{-7}	84	97	-7.07×10^{-2}	1.43×10^{-1}

The shortest CON round trip changes no coordinate. The barrier moves by -5.68×10^{-14} eV and the images by at most 2.66×10^{-15} Å. Each of the 25 harness pairs also changes no coordinate, keeps 56 modes, and finishes in 60 steps at the same printed barrier. Their barrier shift is -2.31×10^{-14} eV and their largest displacement is 3.55×10^{-15} Å. The two floors are both at the level of rebuilding the calculator. The table keeps them separate. Six decimals move the file by 4.90×10^{-7} Å, the barrier by -4.14×10^{-7} eV, and the images by 6.22×10^{-7} Å, and the band still converges in 60 steps with 56 modes. Extended XYZ moves the file by 5.00×10^{-9} Å and the barrier by $+1.09 \times 10^{-9}$ eV, with the same step count and the same mode count. The ASE trajectory matches the language-pair floor. An extra identifier array on that trajectory does not survive the round trip, and the mode count stays 56 because the mask does.

POSCAR moves the file by 4.90×10^{-17} Å and drops the height constraint, so the middle image has 57 free modes. The band converges in 98 steps, 0.160 Å away, with a barrier 7.02×10^{-2} eV lower. That is the 70.2 meV in the abstract. The ASE eon writer produces no file, because a per-axis constraint is not a constraint that writer emits. The ASE eon reader keeps 54 modes, runs to the 4000-step cap without converging, and finishes 0.352 eV and 5.43 Å away. Restoring periodicity by hand leaves the mask collapse in place: 54

modes, 127 steps, a barrier shift of -2.93×10^{-6} eV, and a displacement of 4.21×10^{-2} Å.

The H5MD block in the table is a second measurement, distinct from the survey row. The writer attaches a Boolean $N \times 3$ mask array, an identifier array, the velocity unit Å ps^{-1}, and the boundary periodic, periodic, none. MDAnalysis reading that file, whether it wrote the file or h5py did, returns 84 free modes and a barrier 7.07×10^{-2} eV lower, with a file shift of 4.60×10^{-7} Å. The mode count is the loss of the mask and the identifier. h5py reading the MDAnalysis file keeps 56 modes, finishes in 60 steps, and moves the barrier by -1.98×10^{-7} eV. h5py reading its own file is bit-exact and lands on the language-pair floor. The coordinate gap between the two files is the single-precision write.

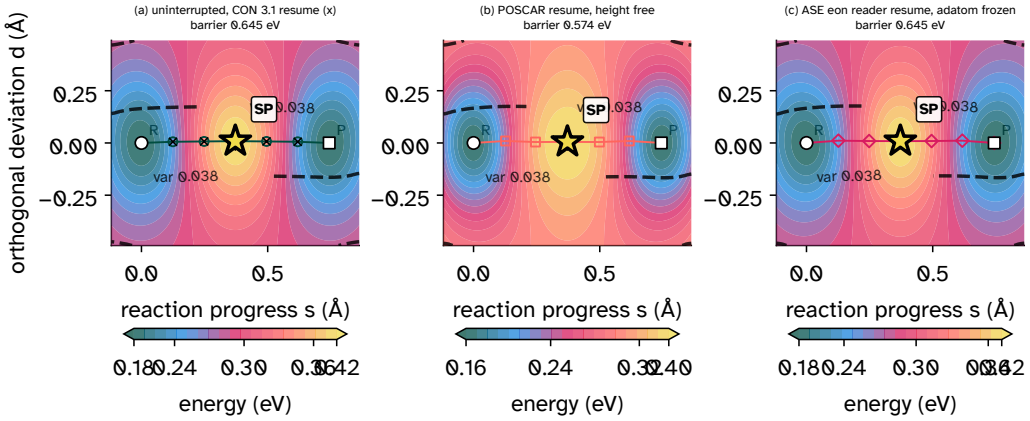


Figure 2: Resumed climbing-image bands of the EMT Pt(100) hop on the (s, d) plane [55]. Each panel is the energy surface of its own band. (a) Uninterrupted, with the CON 3.1 resume as crosses. (b) POSCAR, height constraint dropped. (c) ASE eon reader with periodicity restored by hand. The routes differ in barrier and in free modes, as Table 11 records.

6.1. The mask and the prefactor

The symmetric hop can sit on the straight line, so a lost lateral constraint need not move the barrier by much. The same 28-atom Pt(100) $3 \times 3 \times 3$ slab, seven images in all, was then converged with FIRE to 0.01 eV/Å instead of steepest descent [45]. ASE 3.29.0 has no eOn Morse calculator in this run, so the relaxation, the band and the Hessian use EMT. With the height held, the barrier is 0.36659812266626624 eV on a path of

2.9004016648060382 Å. The middle image has 56 free modes and one imaginary mode, −4.076276454192005 meV, with 0.9872893953755115 of its weight on the adatom's lateral motion. The Vineyard prefactor of that Hessian is $3.2438730218892426 \times 10^{10}$ Hz [2]. FIRE took 26 steps and then 16 climbing steps.

The ASE eon reader turns the height-held column into a full freeze, so the adatom is fixed on every axis. The default interpolation then stops because the constraints change the path, and the band was built with the constraints ignored during interpolation. FIRE took 26 steps and then 14 climbing steps, reached 0.007095 eV/Å, and left the adatom on the straight interpolation. The relative peak is 0.3665836803750313 eV on a path of 2.9595547559012947 Å. The middle image has 54 free modes and no imaginary mode, so the Vineyard ratio is undefined. Figure 3 is the two bands. The barriers differ in the fifth digit. The prefactor exists for one reading and does not exist for the other.

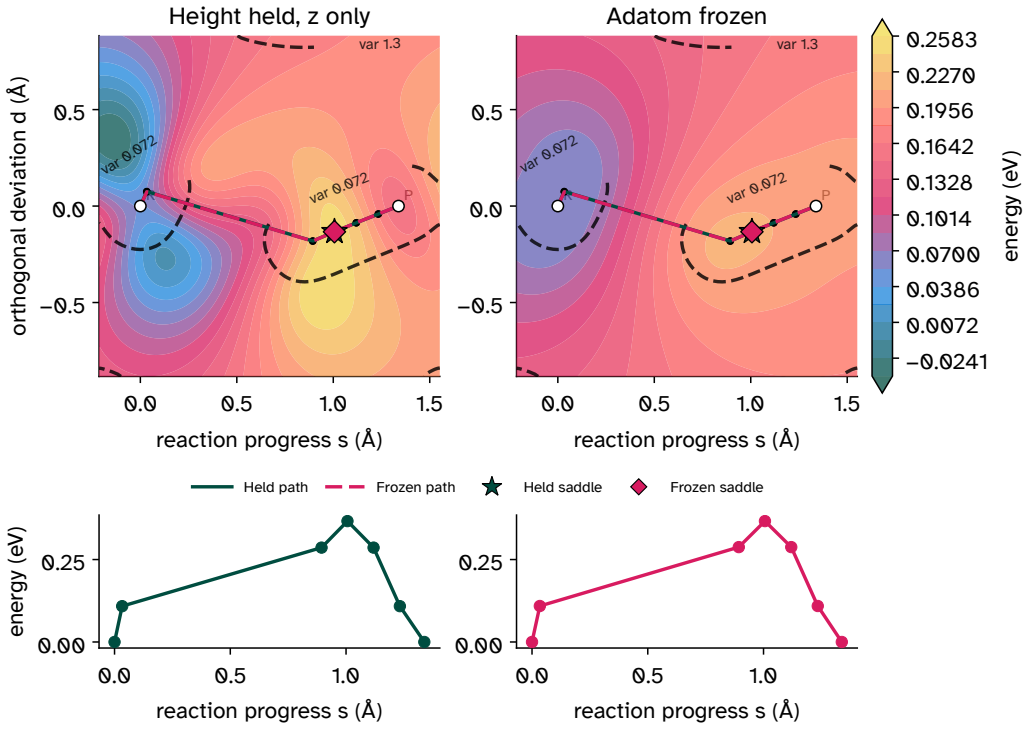


Figure 3: EMT climbing-image bands for a Pt adatom hop on Pt(100), each mask on its own (s, d) map [55]. Height held: 0.366598 eV, 56 free modes, one imaginary mode. ASE eon reading: 0.366584 eV, 54 free modes, no imaginary mode.

6.2. eOn resuming a Morse band

An eOn 3.6.0 client ran a Morse Pt potential on a 28-atom slab, bottom atoms at mask 7, five movable images, steepest descent [18]. The band section selects the optimiser. This run sets steepest descent there. When the total number of optimiser steps matches, the shortest round trip reproduces the uninterrupted barrier, 0.63493 eV, with barrier difference 0 and a largest minimum-image displacement of $8.881784197001252 \times 10^{-15}$ Å. Six decimals give barrier 0.634931 eV and a largest minimum-image displacement of $4.918050526825368 \times 10^{-7}$ Å. Figure 4 is that comparison. Section S1 gives the per-image displacement.

A control file left the default band optimiser in place. Both printers of that control finish at 0.670723 eV. The uninterrupted barrier in the same file is 0.63361 eV, so the difference is 0.037113 eV. The two printers agree with each other. They do not agree with the uninterrupted barrier, because the checkpoint does not carry the optimiser history.

A dimer started from the round-trip geometry stops for the same reason as the uninterrupted dimer, too many iterations. Both report saddle energy −112.618941 eV and curvature −10.613681. Six decimals give −112.618944 eV and −10.613677, an energy shift of -3×10^{-6} eV. The comparison is of the stopped search.

6.3. The velocity unit and identity

On the 28-atom slab the kinetic energy is 0.8878595057627452 eV in the natural unit and 92.0201533142073 eV when those numbers are read as Å/fs. The ratio is 103.64269652680504, the square of 10.180505710759414 fs. One velocity Verlet step of 1 fs then moves the atoms by an RMSD of 0.09556596694568728 Å, maximum 0.31544697368347396 Å [56]. On a separate 10,001-atom slab at 300 K the logged kinetic energy is 390.085819 eV and the value obtained by reading those numbers as Å/fs is 40429.545793 eV. Section S4 plots the step.

The public HCN and HNC endpoints are frames 0 and 1 of one file [57, 58]. On both, readcon-core returns masks `000 000 000`, identifiers `0 1 2`, and the declared natural unit. Frame 2 is a constructed probe, not a public endpoint: masks `000 000 100` and identifiers `2 0 1`, with hydrogen fixed on x only. The ASE eon reader stops on the velocity block of that file. Given the probe coordinates alone, it returns the three atoms as free, free, and fixed on every axis, and it returns no identifiers. Table 12 is that split.

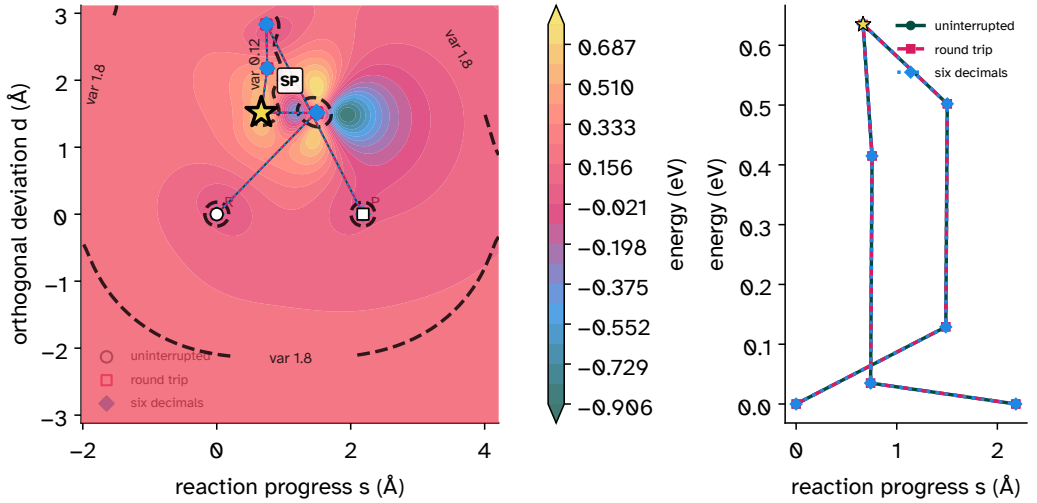


Figure 4: Final images of the Morse band on the (s, d) plane [55]. With the step count matched, the shortest round trip returns the uninterrupted barrier, 0.63493 eV. The colour bar is the fitted surface. The image energies are the energy column of the uninterrupted band, and the highest image is 0.634930 eV.

7. Performance

Timings below are medians. Where an interval is reported, it is a bias-corrected and accelerated 95 percent interval of the median from 9999 resamples [59]. The host, the compiler and the thread pin are part of the result [42, 41]. Parallel runs are compared at fixed work or fixed work per rank. This section names the build that was clocked.

Thread timings and peer campaign A used an Intel(R) Xeon(R) Processor, family 6, model 207, 8 hardware threads, rustc 1.88.0, g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2 24.04.1) and Python 3.12.3. TODO(verify): commercial CPU SKU, clock frequency, RAM and operating-system release for that campaign. Peer campaign B used an Intel(R) Xeon(R) Processor, 8 hardware threads, rustc 1.99.0 and Python 3.12.15. TODO(verify): commercial CPU SKU, family and model, clock frequency, RAM, operating-system release and C compiler for campaign B. The MPI campaign used an Intel(R) Xeon(R) Processor, family 6, model 207, stepping 2, 8 cores, 1 thread per core, 1 socket, 49,342,904 kB of memory, Linux 6.12.94+, rustc 1.99.0, GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2 24.04.1) and MPICH 4.2.3. TODO(verify): commercial CPU SKU, clock frequency and operating-system distribution for the MPI host.

Table 12: Per-atom mask, id and velocity. A mask is three bits in order xyz. The probe stores ids 2, 0 and 1 and fixes hydrogen on x only. The slab rows are one buried atom, one surface atom and the adsorbate.

Atom	Reader	Mask	Id	Velocity
HCN C	readcon-core	000	0	present
HCN N	readcon-core	000	1	present
HCN H	readcon-core	000	2	present
HNC C	readcon-core	000	0	present
HNC N	readcon-core	000	1	present
HNC H	readcon-core	000	2	present
probe C	readcon-core	000	2	present
probe N	readcon-core	000	0	present
probe H	readcon-core	100	1	present
probe C	ASE eon	free	absent	absent
probe N	ASE eon	free	absent	absent
probe H	ASE eon	all-fixed	absent	absent
slab bottom Pt	readcon-core	111	0	present
slab surface Pt	readcon-core	000	9000	present
slab adsorbate H	readcon-core	001	10000	present

The kernel string is recorded. The store campaign used an Intel(R) Xeon(R) Processor, 8 CPUs, 47.07 GiB, Python 3.12.3 and rustc 1.88.0. TODO(verify): commercial CPU SKU, family and model, clock frequency and operating-system release for the store host. None of the logs records a clock frequency, so none is stated.

7.1. Threads

The file is 10,000 copies of one 218-atom frame, 167,710,000 bytes, read by readcon-core 0.17.0 built with rustc 1.88.0. OpenMP and BLAS are pinned to one thread outside the pool under test. Table 13 is the warm-cache read. The native Rust pool finishes in 0.125501 s, 1336 MB/s, 79681 frames/s, which is 12.6 μs per frame. The C++ wrapper eOn calls finishes in 0.13315 s. The median of the paired Rust-to-C++ ratios is 0.904, with interval $[0.815, 0.993]$. That interval belongs to the paired ratios, not to the quotient of the two medians just given. At one thread the medians are 0.483641 s and 0.484879 s, and the median of the paired ratios is 0.991. Building a Python object per frame takes 1.234571 s. One EMT force call on this host has median 5.2623×10^{-5} s, so the native read of the whole file is 2384.9 such calls. The

in-process Morse timing from the band study is 3.036×10^{-4} s per call, and the same file is 413.4 of those calls.

Table 13: Warm read of 10,000 frames, 167,710,000 bytes, readcon-core 0.17.0. Native rows are 15 repeats. The Python row is 9 repeats. The interval is a bias-corrected and accelerated 95 percent interval of the median.

Reader	Median (s)	MB/s	Interval (s)
Rust, automatic pool	0.125501	1336	[0.115354, 0.137084]
C++ wrapper, automatic pool	0.13315	1260	[0.120279, 0.153288]
Rust, 1 thread	0.483641	346.8	[0.473807, 0.486509]
C++ wrapper, 1 thread	0.484879	345.9	[0.480843, 0.537422]
Python, full objects	1.234571	135.8	[1.216353, 1.303821]

A separate five-repeat sweep on the same file sets the pool size. Rust goes from 0.50026 s at one thread to 0.11503 s at eight, a speedup of 4.35. C++ goes from 0.50177 s to 0.12793 s, a speedup of 3.92. Python full objects go from 1.2755 s to 1.1070 s, a speedup of 1.15. A second Python sweep, thirty interleaved pairs on the same file, is a different campaign. Its eight-thread speedup is 1.284, with interval $[1.263, 1.313]$. Section S14 records that sweep. Hashes of the returned fields at 1, 2, 3, 4 and 8 threads are identical. Figure 5 is the automatic-pool rates. Figure 6 places the five-repeat scaling rates under the memory-copy rate. No Amdahl curve is drawn. The in-process file read is 0.094481528 s and the boundary scan is 0.034419954 s. Their sum, 0.128901482 s, is 0.266523 of the 15-repeat one-thread median 0.483640922 s. That quotient is not a serial fraction of the five-repeat sweep.

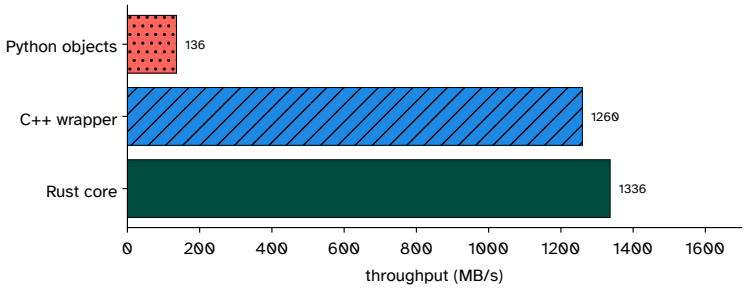


Figure 5: Warm-cache rate for the automatic Rust pool, the C++ wrapper and the Python full-object read, from the medians in Table 13.

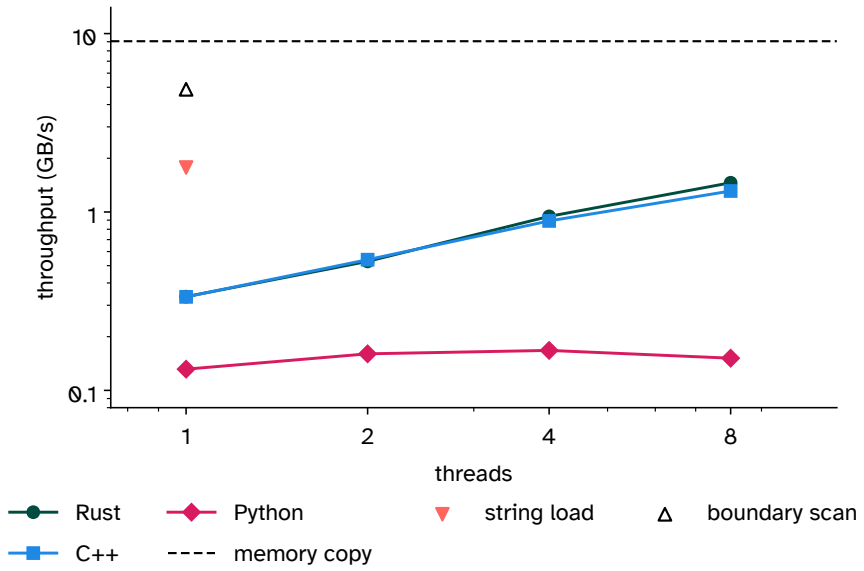


Figure 6: Five-repeat scaling rates for Rust, C++ and Python on readcon-core 0.17.0. The dashed line is the hot-cache copy, 9.06 GB/s. The triangles are the 15-repeat file-read and boundary-scan rates. They are not an Amdahl curve for the scaling points.

7.2. Peers

Table 14 is readcon-core through its Python binding at one worker, divided by the peer, so a ratio below 1 means readcon-core finished sooner. Campaign A, readcon-core 0.17.0 and rustc 1.88.0, thirty interleaved pairs, compares the ASE eon reader, chemfiles XYZ, h5py and MDAnalysis H5MD. On 10,000 frames the ratios are 0.337 against ASE eon (readcon-core 1.269 s, ASE 3.745 s), 2.15 against a chemfiles read of the XYZ coordinates, 67.1 against h5py when h5py returns one numeric array (peer 0.01899 s), and 0.207 against MDAnalysis H5MD (peer 6.179 s). The CON reader returns the mask, the identifier, the cell and the periodic flags. The 67.1 ratio is that full frame against the numeric array, which is what Table 14 records in its fields column. Campaign B, rustc 1.99.0 and Python 3.12.15, adds ASE extended XYZ and the ASE trajectory. The ASE eon ratio on the same 10,000-frame file is 0.348, against 0.337 in campaign A. Absolute readcon-core times differ between the campaigns, near 2.0 s in B and 1.27 s in A, so only the reproduced ratio is compared. On that long file the ASE trajectory is level with readcon-core (ratio 1.02). On one slab frame the ASE trajectory is faster (ratio 3.19). Section S9 extends the table to the 1-frame and 100-frame files. On one frame

the open dominates, and h5py and chemfiles lose the lead they have on the long file.

Table 14: Paired ratios of the one-worker Python read to each peer, readcon-core time over peer time, so a value below 1 means readcon-core finished sooner. Each cell is 30 interleaved pairs; the interval is a bias-corrected and accelerated 95 percent interval of the median. The fields column says what the peer returns. The two ASE rows beyond the eon reader come from a second campaign on a shared host, in which the ASE eon ratio on the 10000-frame file reproduced at 0.348 against 0.337.

Payload	Peer	Fields	Ratio	Interval
10000 frames	ASE eon	same CON text	0.337	[0.335, 0.345]
10000 frames	chemfiles XYZ	coordinates	2.15	[2.14, 2.17]
10000 frames	h5py	one binary array	67.1	[66.5, 67.3]
10000 frames	MDAnalysis H5MD	coordinates	0.207	[0.204, 0.213]
Pt slab	ASE eon	same CON text	0.233	[0.231, 0.241]
Pt slab	chemfiles XYZ	coordinates	1.14	[1.13, 1.16]
Pt slab	h5py	one binary array	6.05	[5.91, 6.27]
Pt slab	MDAnalysis H5MD	coordinates	0.394	[0.387, 0.404]
10000 frames	ASE extended XYZ	ASE atoms	0.392	[0.338, 0.42]
10000 frames	ASE trajectory	ASE atoms	1.02	[0.948, 1.08]
Pt slab	ASE extended XYZ	ASE atoms	0.184	[0.18, 0.192]
Pt slab	ASE trajectory	ASE atoms	3.19	[3.08, 3.28]

7.3. MPI

The MPI campaign used readcon-core 0.17.0 and readcon-db 0.1.8 on the eight-core host named above. Ranks 16 and 32 were not run. MPICH 4.2.3 used the ch4:ofi device, bound to cores, with UCX and libfabric on TCP. Each rank is one thread. Each cell is five repeats. Cold repeats drop the page cache. Warm repeats follow an untimed warmup. The median is the third of the five sorted times. Every one of the 480 repeats matched the serial bytes.

Strong scaling holds the file fixed. The CON file is 1,583,691 copies of a small frame, 3,167,382 frames, 1,073,742,498 bytes. The database file is 30,965 frames of 1,000 atoms, ingested four times, with four shards on the sharded path. Table 15 is the warm medians. Shared read goes from 9.088 s to 1.504 s, a ratio 6.04. Scatter goes from 9.223 s to 1.836 s, ratio 5.02. MPI-IO write goes from 8.369 s to 1.416 s, ratio 5.91. Gather write goes from 9.490 s to 2.152 s, ratio 4.41. One-writer ingest goes from 31.699 s to 16.810 s, ratio 1.89. Four-shard ingest goes from 32.549 s to 18.805 s, ratio 1.73. Cold shared

read goes from 9.904 s to 2.113 s. Figure 7 is the warm speedup. The full rank list, cold and warm, is in Section S12.

Table 15: Strong scaling, warm page cache. Each cell is the median of five repeats. t_1/t_8 is the one-rank median divided by the eight-rank median. The file is 1,073,742,498 bytes. readcon-core holds 3,167,382 frames. readcon-db ingests four copies of a 1,074,071,389-byte trajectory (30,965 frames of 1,000 atoms), batch 4096. One thread per rank. Every output matches the serial bytes.

Path	1 rank (s)	8 ranks (s)	min–max at 8	t_1/t_8
read, shared file	9.088	1.504	1.416–1.570	6.04
read, scatter	9.223	1.836	1.810–1.875	5.02
write, MPI-IO	8.369	1.416	1.366–1.491	5.91
write, gather	9.490	2.152	2.102–2.262	4.41
ingest, one writer	31.699	16.810	16.663–17.330	1.89
ingest, four shards	32.549	18.805	18.644–18.879	1.73

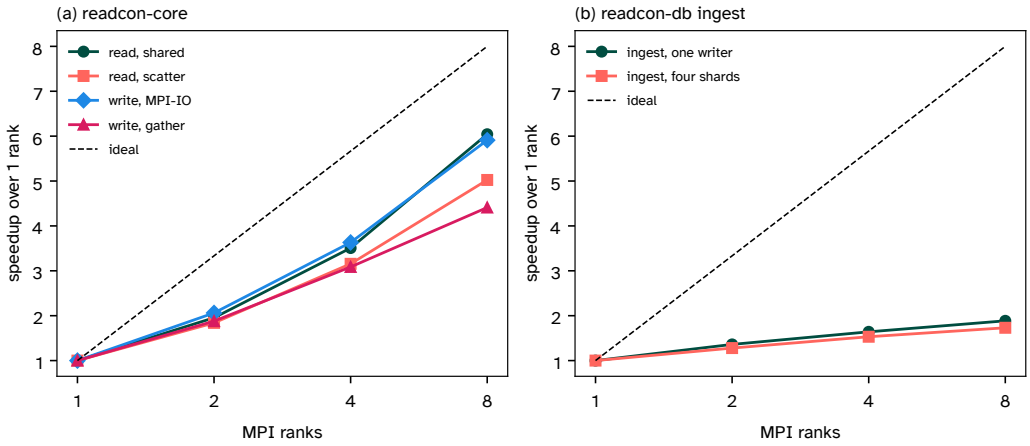


Figure 7: Strong-scaling speedup from the warm medians in Table 15. The dashed line is ideal speedup. Ranks 1, 2, 4 and 8 on the 8-core host. Every output matched the serial bytes.

Weak scaling gives every rank the same frame count as the one-rank run. readcon-core holds 400,000 frames per rank. readcon-db ingests 7,700 frames of 1,000 atoms per rank, four copies. Table 17 is the warm medians, and the ratio is the eight-rank time over the one-rank time. Shared read grows by 1.36, scatter by 1.61, MPI-IO write by 1.33, and gather write by 1.78. One-writer

ingest grows by 3.77, from 7.841 s to 29.594 s. Four-shard ingest grows by 4.68, from 7.465 s to 34.899 s. Every rank writes one virtio ext4 device, and the byte count grows with the rank count. Table 16 is the aggregate rate on that device. Shared read and MPI-IO write still gain rate from one rank to eight. Both ingest paths level off, and at eight ranks the sharded path is the slower of the two on the same bytes. Figure 8 is that growth.

Table 16: Warm weak scaling on one virtio ext4 device. The aggregate rate is `bytes_total` divided by the median, and a megabyte is 10^6 bytes. The ratio is the median at that rank count divided by the one-rank median of the same path. Shared read and MPI-IO write are the endpoints. Both ingest paths are the full rank list, which is where the rate flattens.

Path	Ranks	Time (s)	Bytes	MB/s	$t(R)/t(1)$
shared read	1	1.161	135,600,000	116.8	1.00
shared read	8	1.5815	1,084,800,000	685.9	1.36
MPI-IO write	1	1.0369	135,600,000	130.8	1.00
MPI-IO write	8	1.3821	1,084,800,000	784.9	1.33
one-writer ingest	1	7.841	1,068,323,900	136.2	1.00
one-writer ingest	8	29.594	8,546,831,576	288.8	3.77
four-shard ingest	1	7.465	1,068,323,900	143.1	1.00
four-shard ingest	2	10.856	2,136,675,340	196.8	1.45
four-shard ingest	4	18.375	4,273,391,820	232.6	2.46
four-shard ingest	8	34.899	8,546,831,576	244.9	4.68

Table 17: Weak scaling, warm page cache. Each rank holds the same frame count as the one-rank run. t_8/t_1 is the eight-rank median divided by the one-rank median. readcon-core holds 400,000 frames per rank (135,600,000 bytes at one rank). readcon-db ingests 7,700 frames of 1,000 atoms per rank, four copies (30,800 frames per rank). Medians of five repeats. Every output matches the serial bytes.

Path	1 rank (s)	8 ranks (s)	min–max at 8	t_8/t_1
read, shared file	1.161	1.581	1.573–1.792	1.36
read, scatter	1.218	1.962	1.949–2.264	1.61
write, MPI-IO	1.037	1.382	1.365–1.446	1.33
write, gather	1.211	2.161	2.097–2.296	1.78
ingest, one writer	7.841	29.594	28.943–30.364	3.77
ingest, four shards	7.465	34.899	34.807–36.533	4.68

7.4. The store

The store measurements used readcon-db 0.1.8 and readcon-core 0.16.2 [60] on the store host named above. The archived store release is 0.2.0 [54].

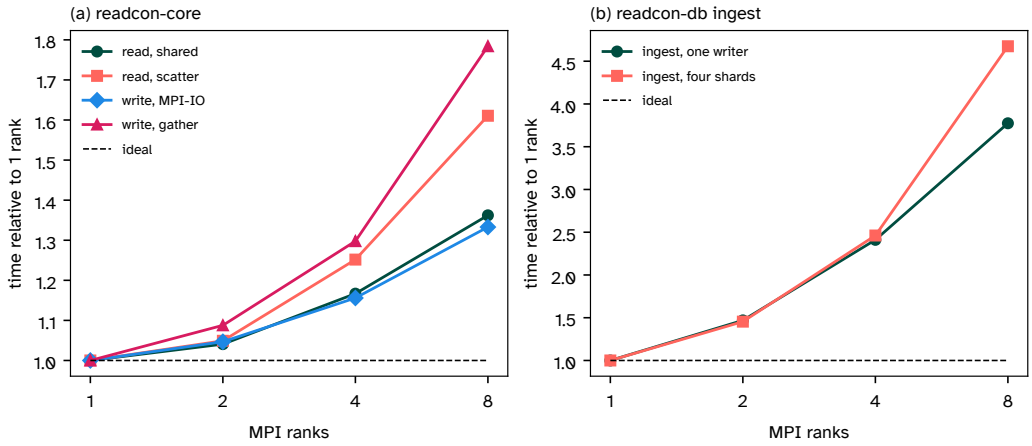


Figure 8: Weak scaling from the warm medians in Table 17. The dashed line is a ratio of 1, the same time as one rank. The ingest ratios lie above that line.

Threads are pinned to 1. Five repeats are interleaved. The page cache was not dropped. A fresh-process query starts a new process with the cache left in place. A warm query is the next query in that process. The timed corpora have 1,000 and 10,000 frames, and every frame has its own xxHash3-128. The frames are ASE effective-medium geometries stored as CON text. Section S10 records the repeats and the figures.

At 10,000 frames the ingest medians are 0.311306 s for readcon-db (32,123 frames/s, 35.09 MB/s), 48.411 s for ASE's database committing every write (206.6 frames/s), and 5.985 s for one ASE connection that still commits every 5,000 rows (1,671 frames/s). The paired ratios are 0.006800 against the per-write commits and 0.05283 against the single connection. At 10,000 frames the readcon-db fresh-process medians are 8.14 ms, 10.1 ms and 9.57 ms for composition, energy and maximum force, and the warm medians are 0.176 ms, 0.211 ms and 0.208 ms. ASE's database, on the same three queries, takes 0.286 s, 0.223 s and 0.225 s in a fresh process and 0.258 s, 0.203 s and 0.206 s warm. A directory of the same text takes 0.140 s, 0.138 s and 0.138 s warm. readcon-db occupies 2.503 times the raw CON text: the text plus the indexes. The map used for these tables starts at 2 GiB, so a one-million-frame open exceeds it and stays outside the tables.

8. Conclusions

The geometry file in a rare-event campaign is the restart state of the search. The resume reads a per-axis mask, a persistent identifier, a declared time unit, binary64 coordinates with their periodic flags, and a version two readers agree on. On one masked slab, every common format tested here drops at least one of those fields. The ASE POSCAR resume drops the height constraint and finishes 70.2 meV lower. Five extended XYZ readers return four answers for one file. CON 3.1 states the grammar and the corpus decides the listed files. The conformance goldens are Python, C, Fortran and Julia. The resume harness runs the same parser from Python, C, C++, Fortran and Julia, and its twenty-five writer-reader pairs finish at the uninterrupted barrier of 0.644561366 eV. With the height held, the Vineyard prefactor of the EMT hop is $3.2438730218892426 \times 10^{10}$ Hz. The ASE eon reading of the same band has no imaginary mode, so that ratio is undefined. A warm read of 10,000 frames of 218 atoms takes 12.6 μs per frame on the 0.17.0 build named in Section 7. On that file a chemfiles read of the XYZ coordinates is 2.15 times as fast, and h5py is 67.1 times as fast when the timed object is one numeric array. Shared MPI reads speed up by 6.04 from one rank to eight on this host. Under weak scaling the four-shard ingest grows by 4.68 because every rank writes the same device.

Data availability

The release version and its DOI are TODO(verify: 0.17.2 pin). The concept record is `https://doi.org/10.5281/zenodo.18753567` [52]. readcon-core 0.16.2, the reader used for the store timings, is `https://doi.org/10.5281/zenodo.23264985` [60]. The companion store is archived at `https://doi.org/10.5281/zenodo.23652386` [54]. The specification, the grammar, the corpus and the campaign results are the supplementary material.

CRediT authorship contribution statement

Rohit Goswami: Conceptualization, Methodology, Software, Visualization, Writing – original draft, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The author's family, and the pets who share the house, carried the ordinary days while this was written. Ruhila Goswami designed the colour palette used for every figure.

References

[1] H. Eyring, The activated complex in chemical reactions, The Journal of Chemical Physics 3 (2) (1935) 107–115. doi:10.1063/1.1749604.
URL https://pubs.aip.org/jcp/article/3/2/107/203352/The-Activated-Complex-in-Chemical-Reactions

[2] G. H. Vineyard, Frequency factors and isotope effects in solid state rate processes, Journal of Physics and Chemistry of Solids 3 (1-2) (1957) 121–127. doi:10.1016/0022-3697(57)90059-8.
URL https://linkinghub.elsevier.com/retrieve/pii/0022369757900598

[3] P. Hänggi, P. Talkner, M. Borkovec, Reaction-rate theory: fifty years after Kramers, Reviews of Modern Physics 62 (2) (1990) 251–341. doi:10.1103/revmodphys.62.251.
URL https://link.aps.org/doi/10.1103/RevModPhys.62.251

[4] H. Jónsson, G. Mills, K. W. Jacobsen, Nudged elastic band method for finding minimum energy paths of transitions, in: Classical and Quantum Dynamics in Condensed Phase Simulations, WORLD SCIENTIFIC, 1998, pp. 385–404. doi:10.1142/9789812839664_0016.
URL http://www.worldscientific.com/doi/abs/10.1142/9789812839664_0016

[5] G. Henkelman, B. Uberuaga, H. Jónsson, A climbing image nudged elastic band method for finding saddle points and minimum energy paths, The Journal of Chemical Physics 113 (22) (2000) 9901–9904.

doi:10.1063/1.1329672.
URL https://pubs.aip.org/jcp/article/113/22/9901/185050/A-climbing-image-nudged-elastic-band-method-for

[6] G. Henkelman, H. Jónsson, Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points, The Journal of Chemical Physics 113 (22) (2000) 9978–9985. doi:10.1063/1.1323224.
URL https://pubs.aip.org/jcp/article/113/22/9978/184858/Improved-tangent-estimate-in-the-nudged-elastic

[7] V. Ásgeirsson, B. O. Birgisson, R. Bjornsson, U. Becker, F. Neese, C. Riplinger, H. Jónsson, Nudged elastic band method for molecular reactions using energy-weighted springs combined with eigenvector following, Journal of Chemical Theory and Computation 17 (8) (2021) 4929–4945. doi:10.1021/acs.jctc.1c00462.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.1c00462

[8] G. Henkelman, H. Jónsson, A dimer method for finding saddle points on high dimensional potential surfaces using only first derivatives, The Journal of Chemical Physics 111 (15) (1999) 7010–7022. doi:10.1063/1.480097.
URL https://pubs.aip.org/jcp/article/111/15/7010/475160/A-dimer-method-for-finding-saddle-points-on-high

[9] J. Kästner, P. Sherwood, Superlinearly converging dimer method for transition state search, The Journal of Chemical Physics 128 (1) (2008) 014106. doi:10.1063/1.2815812.
URL https://pubs.aip.org/jcp/article/128/1/014106/928095/Superlinearly-converging-dimer-method-for

[10] R. A. Olsen, G. J. Kroes, G. Henkelman, A. Arnaldsson, H. Jónsson, Comparison of methods for finding saddle points without knowledge of the final states, The Journal of Chemical Physics 121 (20) (2004) 9776–9792. doi:10.1063/1.1809574.
URL https://pubs.aip.org/jcp/article/121/20/9776/532635/Comparison-of-methods-for-finding-saddle-points

[11] W. E, W. Ren, E. Vanden-Eijnden, String method for the study of rare events, Physical Review B 66 (5) (2002) 052301. doi:10.1103/physre

vb.66.052301.
URL https://link.aps.org/doi/10.1103/PhysRevB.66.052301

[12] B. Peters, A. Heyden, A. T. Bell, A. Chakraborty, A growing string method for determining transition states: Comparison to the nudged elastic band and string methods, The Journal of Chemical Physics 120 (17) (2004) 7877–7886. doi:10.1063/1.1691018.
URL https://pubs.aip.org/jcp/article/120/17/7877/534771/A-growing-string-method-for-determining-transition

[13] G. Henkelman, H. Jónsson, Long time scale kinetic Monte Carlo simulations without lattice approximation and predefined event table, The Journal of Chemical Physics 115 (21) (2001) 9657–9666. doi:10.1063/1.1415500.
URL https://pubs.aip.org/jcp/article/115/21/9657/453229/Long-time-scale-kinetic-Monte-Carlo-simulations

[14] L. Xu, G. Henkelman, Adaptive kinetic Monte Carlo for first-principles accelerated dynamics, The Journal of Chemical Physics 129 (11) (2008) 114104. doi:10.1063/1.2976010.
URL https://pubs.aip.org/jcp/article/129/11/114104/296009/Adaptive-kinetic-Monte-Carlo-for-first-principles

[15] A. F. Voter, F. Montalenti, T. C. Germann, Extending the time scale in atomistic simulation of materials, Annual Review of Materials Research 32 (1) (2002) 321–346. doi:10.1146/annurev.matsci.32.112601.141541.
URL https://www.annualreviews.org/doi/10.1146/annurev.matsci.32.112601.141541

[16] D. Perez, E. D. Cubuk, A. Waterland, E. Kaxiras, A. F. Voter, Long-time dynamics through parallel trajectory splicing, Journal of Chemical Theory and Computation 12 (1) (2015) 18–28. doi:10.1021/acs.jctc.5b00916.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.5b00916

[17] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time simulations of atomic scale systems, Modelling and Simulation in Materials Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055

002.
URL https://iopscience.iop.org/article/10.1088/0965-0393/22/5/055002

[18] R. Goswami, S. Chill, R. Terrell, G. Henkelman, M. Welborn, L. Zhang, A. Pedersen, t brink, E. Edelmann, J. Claude, Alejandro, S. H. Jung, S. A. Ghasemi, chemist29, satishskamath, Maxim, T. Dijkhuis, via9a, TheochemUI/eOn: v3.6.0, Zenodo, software, version 3.6.0 (2026). doi:10.5281/zenodo.23166734.

[19] D. P. Anderson, BOINC: A system for public-resource computing and storage, in: Fifth IEEE/ACM International Workshop on Grid Computing, IEEE, 2004, pp. 4–10. doi:10.1109/grid.2004.14.
URL http://ieeexplore.ieee.org/document/1382809/

[20] A. Pedersen, H. Jónsson, Distributed implementation of the adaptive kinetic Monte Carlo method, Mathematics and Computers in Simulation 80 (7) (2010) 1487–1498. doi:10.1016/j.matcom.2009.02.010.
URL https://linkinghub.elsevier.com/retrieve/pii/S0378475409000500

[21] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin, E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Journal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648x/aa680e.
URL https://iopscience.iop.org/article/10.1088/1361-648X/aa680e

[22] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Sella, an open-source automation-friendly molecular saddle point optimizer, Journal of Chemical Theory and Computation 18 (11) (2022) 6974–6988. doi:10.1021/acs.jctc.2c00395.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.2c00395

[23] G. Kresse, J. Furthmüller, Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set, Computational Materials Science 6 (1) (1996) 15–50. doi:10.1016/0927-0256(96)00008-0.
URL https://linkinghub.elsevier.com/retrieve/pii/0927025696000080

[24] E. Tiesinga, P. J. Mohr, D. B. Newell, B. N. Taylor, CODATA recommended values of the fundamental physical constants: 2018, Reviews of Modern Physics 93 (2) (2021) 025010. doi:10.1103/revmodphys.93.025010.
URL https://link.aps.org/doi/10.1103/RevModPhys.93.025010

[25] IEEE Computer Society, IEEE standard for floating-point arithmetic (2019). doi:10.1109/ieeestd.2019.8766229.
URL https://ieeexplore.ieee.org/document/8766229/

[26] W. D. Clinger, How to read floating point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 92–101. doi:10.1145/93542.93557.
URL http://portal.acm.org/citation.cfm?doid=93542.93557

[27] G. L. Steele, J. L. White, How to print floating-point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 112–126. doi:10.1145/93542.93559.
URL http://portal.acm.org/citation.cfm?doid=93542.93559

[28] U. Adams, Ryū: fast float-to-string conversion, in: Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2018, pp. 270–282. doi:10.1145/3192366.3192369.
URL https://dl.acm.org/doi/10.1145/3192366.3192369

[29] P. d. Buyl, P. H. Colberg, F. Höfling, H5MD: A structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465514000447

[30] M. Folk, G. Heber, Q. Koziol, E. Pourmal, D. Robinson, An overview of the HDF5 technology suite and its applications, in: Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases, ACM, 2011, pp. 36–47. doi:10.1145/1966895.1966900.
URL https://dl.acm.org/doi/10.1145/1966895.1966900

[31] D. Spångberg, D. S. D. Larsson, D. v. d. Spoel, Trajectory NG: portable, compressed, general molecular dynamics trajectories, Journal of Molecular Modeling 17 (10) (2011) 2669–2685. doi:10.1007/s00894-010-0948-5.
URL http://link.springer.com/10.1007/s00894-010-0948-5

[32] M. Lundborg, R. Apostolov, D. Spångberg, A. Gärdenäs, D. van der Spoel, E. Lindahl, An efficient and extensible format, library, and API for binary trajectory data from molecular simulations, Journal of Computational Chemistry 35 (3) (2013) 260–269. doi:10.1002/jcc.23495.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.23495

[33] R. Rew, G. Davis, NetCDF: an interface for scientific data access, IEEE Computer Graphics and Applications 10 (4) (1990) 76–82. doi:10.1109/38.56302.
URL http://ieeexplore.ieee.org/document/56302/

[34] G. Fraux, L. Kimms, J. Fine, G. P. Barletta, M. Dimura, F. Coudert, Pelsa, M. Levesque, S. Maiti, S. Guionniere, Jmintser, chemfiles/chemfiles: Version 0.10.4, Zenodo, software, version 0.10.4 (2023). doi:10.5281/zenodo.7904565.

[35] R. Gowers, M. Linke, J. Barnoud, T. Reddy, M. Melo, S. Seyler, J. Domański, D. Dotson, S. Buchoux, I. Kenney, O. Beckstein, MDAnalysis: A Python package for the rapid analysis of molecular dynamics simulations, in: Proceedings of the Python in Science Conference, SciPy, 2016, pp. 98–105. doi:10.25080/majora-629e541a-00e.
URL https://doi.curvenote.com/10.25080/Majora-629e541a-00e

[36] N. M. O'Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, G. R. Hutchison, Open Babel: An open chemical toolbox, Journal of Cheminformatics 3 (1) (2011) 33. doi:10.1186/1758-2946-3-33.
URL https://jcheminf.biomedcentral.com/articles/10.1186/1758-2946-3-33

[37] T. Verstraelen, W. Adams, L. Pujal, A. Tehrani, B. D. Kelly, L. Macaya, F. Meng, M. Richer, R. Hernández-Esparza, X. D. Yang, M. Chan, T. D. Kim, M. Cools-Ceuppens, V. Chuiko, E. Vöhringer-Martinez, P. W. Ayers, F. Heidar-Zadeh, IOData: A Python library for reading, writing, and converting computational chemistry file formats and generating input files, Journal of Computational Chemistry 42 (6) (2020) 458–464. doi:10.1002/jcc.26468.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.26468

[38] M. Andrysco, R. Jhala, S. Lerner, Printing floating-point numbers: a faster, always correct method, in: Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, ACM, 2016, pp. 555–567. doi:10.1145/2837614.2837654.
URL https://dl.acm.org/doi/10.1145/2837614.2837654

[39] F. Loitsch, Printing floating-point numbers quickly and accurately with integers, in: Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2010, pp. 233–243. doi:10.1145/1806596.1806623.
URL https://dl.acm.org/doi/10.1145/1806596.1806623

[40] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Review 59 (1) (2017) 65–98. doi:10.1137/141000671.
URL https://epubs.siam.org/doi/10.1137/141000671

[41] T. Hoefler, R. Belli, Scientific benchmarking of parallel computing systems, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, ACM, 2015, pp. 1–12. doi:10.1145/2807591.2807644.
URL https://dl.acm.org/doi/10.1145/2807591.2807644

[42] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 international symposium on memory management, ACM, 2013, pp. 63–74. doi:10.1145/2491894.2464160.
URL https://dl.acm.org/doi/10.1145/2491894.2464160

[43] J. Nocedal, Updating quasi-Newton matrices with limited storage, Mathematics of Computation 35 (151) (1980) 773–782. doi:10.1090/s0025-5718-1980-0572855-7.

URL https://www.ams.org/mcom/1980-35-151/S0025-5718-1980-0572855-7/

[44] D. Sheppard, R. Terrell, G. Henkelman, Optimization methods for finding minimum energy paths, The Journal of Chemical Physics 128 (13) (2008) 134106. doi:10.1063/1.2841941.
URL https://pubs.aip.org/jcp/article/128/13/134106/977389/Optimization-methods-for-finding-minimum-energy

[45] E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, P. Gumbsch, Structural relaxation made simple, Physical Review Letters 97 (17) (2006) 170201. doi:10.1103/physrevlett.97.170201.
URL https://link.aps.org/doi/10.1103/PhysRevLett.97.170201

[46] R. Gowers, O. Beckstein, M. Linke, I. Alibay, J. Barnoud, T. Reddy, H. MacDermott-Opeskin, R. Meli, L. Wang, U. Bansal, J. Zeman, J. Detlefs, D. L. Dotson, M. N. Melo, M. Bieniek, ayushsuhane, S. Buchoux, M. Tiberti, N. Michaud-Agrawal, D. Cruz, S. Seyler, denniej0 2, P. Loche, J. Borreguero, Y. Zhuang, T. Yelgoe, N. Pal, Ninad, J. Domański, Z. Wu, MDAnalysis/mdanalysis: Release 2.10.0 of MDAnalysis and MDAnalysisTests, Zenodo, software, version package-2.10.0 (2025). doi:10.5281/zenodo.17382469.

[47] G. Kresse, J. Furthmüller, Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set, Physical Review B 54 (16) (1996) 11169–11186. doi:10.1103/physrevb.54.11169.
URL https://link.aps.org/doi/10.1103/PhysRevB.54.11169

[48] T. Bray, The JavaScript object notation (JSON) data interchange format, Tech. Rep. 8259, IETF (2017). doi:10.17487/rfc8259.
URL https://www.rfc-editor.org/info/rfc8259

[49] H. Bekker, Unification of box shapes in molecular simulations, Journal of Computational Chemistry 18 (15) (1997) 1930–1942. doi:10.1002/(sici)1096-987x(19971130)18:15<1930::aid-jcc8>3.0.co;2-p.
URL https://onlinelibrary.wiley.com/doi/10.1002/(SICI)1096-987X(19971130)18:15<1930::AID-JCC8>3.0.CO;2-P

[50] P. Deutsch, GZIP file format specification version 4.3, Tech. Rep. 1952, IETF (1996). doi:10.17487/rfc1952.
URL https://www.rfc-editor.org/info/rfc1952

[51] Y. Collet, Zstandard compression and the 'application/zstd' media type, Tech. Rep. 8878, IETF (2021). doi:10.17487/rfc8878.
URL https://www.rfc-editor.org/info/rfc8878

[52] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, all versions (2026). doi:10.5281/zenodo.18753567.

[53] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.
URL https://www.nature.com/articles/s41586-020-2649-2

[54] R. Goswami, readcon-db: mmap CON campaign store, Zenodo, software, version 0.2.0. The store timings in the paper used 0.1.8, which has no separate archive. (2026). doi:10.5281/zenodo.23652386.

[55] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.
URL https://linkinghub.elsevier.com/retrieve/pii/S2215016126000683

[56] W. C. Swope, H. C. Andersen, P. H. Berens, K. R. Wilson, A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters, The Journal of Chemical Physics 76 (1) (1982) 637–649. doi:10.1063/1.442716.
URL https://pubs.aip.org/jcp/article/76/1/637/397800/A-computer-simulation-method-for-the-calculation

[57] T. van Mourik, G. J. Harris, O. L. Polyansky, J. Tennyson, A. G. Császár, P. J. Knowles, Ab initio global potential, dipole, adiabatic, and relativistic correction surfaces for the HCN–HNC system, The Journal of Chemical Physics 115 (8) (2001) 3706–3718. doi:10.1063/1.1383586.

URL https://pubs.aip.org/jcp/article/115/8/3706/463154/Ab-initio-global-potential-dipole-adiabatic-and

[58] R. Goswami, Reproducible orchestration of best practices for reaction path optimization with the nudged elastic band, MethodsX 16 (2026) 103899. doi:10.1016/j.mex.2026.103899.
URL https://linkinghub.elsevier.com/retrieve/pii/S2215016126001172

[59] B. Efron, Better bootstrap confidence intervals, Journal of the American Statistical Association 82 (397) (1987) 171–185. doi:10.1080/01621459.1987.10478410.
URL http://www.tandfonline.com/doi/abs/10.1080/01621459.1987.10478410

[60] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, version 0.16.2 (2026). doi:10.5281/zenodo.23264985.